

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Transactional Memory (Part 3)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan and Natalie Enright Jerger

Last Lecture

- Eager and lazy version management
- Eager and lazy conflict detection

This Lecture

- SW and HW transactional memory
- TM challenges
- Sequential consistency

Address-Based Conflict Detection

Read Set: List of shared memory addresses read in transaction

Write Set: List of shared memory addresses modified in transaction

- Constructed on-the-fly (i.e., addresses not known a priori)
- Can be any granularity (e.g., words/objects in SW, blocks in HW)
- False positives are okay (e.g., bloom filters [D. Sanchez et al., MICRO 2007])

Value-Based Conflict Detection

Read Set: List of shared memory **values** read in transaction

Write Set: List of shared memory **values** modified in transaction

- Constructed on-the-fly (i.e., **values** not known a priori)
- Can be any granularity (e.g., words/objects in SW, blocks in HW)
- **Silent stores** are okay (e.g., **approximate silent stores [J. Sreeram and S. Pande, IISWC 2010]**)



Weak and Strong Atomicity

Weak Atomicity:

- No guarantees between transaction and non-transaction code
- Typical for software transactional memory

Strong Atomicity:

- Transaction and non-transaction code regions are kept isolated
- Hardware transactional memory more likely to provide this

Software Transactional Memory

Version Management:

- Eager or lazy, similar overhead of accessing internal structures and memory
- Unbounded undo logging and write buffering in software

Conflict Detection:

- Usually lazy, avoid high overhead of checking for conflicts
- Flexible granularity (e.g., word, block, object)

Commit:

- Global commit lock or many fine-grained locks

Hardware Transactional Memory

Version Management:

- Eager or lazy, HW structures for buffering/logging
- Can leverage write-back cache for lazy VM

Conflict Detection:

- Usually eager, leverage cache coherence protocol
- Add transaction read/write bits to blocks in cache
- Coherence conflict if: 1) read/write to written block, 2) write to read block

Commit:

- Atomically clear all read/write bits

Software vs. Hardware Transactional Memory

Software TM

- Slow software structures
- High instruction overhead
- Relies on explicit locking (deadlocks)
- Strong atomicity is difficult

VS

Hardware TM

- Less portable
- Fixed block granularity
- Relies on coherence implementation
- Limited to cache size (else overflow)

TM Challenges

```
... = *x;  
... = *y;  
*z = 2;  
work(...);  
if (...) {...}  
else {...}  
...
```

TM Challenges

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

coarse-grained; simple for programmers

TM Challenges

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

**Assume eager version management
with lazy conflict detection
(i.e., most optimistic configuration)**

TM Challenges

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

```
atomic {  
    *x = 1;  
    *y = 1;  
    work(...);  
    ...  
}
```

TM Challenges

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

read-write conflict

```
atomic {  
    *x = 1;  
    *y = 1;  
    work(...);  
    ...  
}
```

TM Challenges – Zombie Transactions

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

read-write conflict

zombie transaction

```
atomic {  
    *x = 1;  
    *y = 1;  
    work(...);  
    ...  
}
```

TM Challenges – Zombie Transactions

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

read-write conflict

zombie transaction

```
atomic {  
    *x = 1;  
    *y = 1;  
    work(...);  
    ...  
}
```

Zombie Transactions:

- Conflicted transaction (i.e., doomed to fail) not yet detected
- Due to lazy conflict detection
- Worsened by long transactions

TM Challenges – Zombie Transactions

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

read-write conflict

zombie transaction

```
atomic {  
    *x = 1;  
    *y = 1;  
    work(...);  
    ...  
}
```

```
atomic {  
    ... = *z;  
    ...  
    work(...);  
    ...  
}
```

Zombie Transactions:

- Conflicted transaction (i.e., doomed to fail) not yet detected
- Due to lazy conflict detection
- Worsened by long transactions

TM Challenges – Zombie Transactions

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

read-write conflict

zombie transaction

```
atomic {  
    *x = 1;  
    *y = 1;  
    work(...);  
    ...  
}
```

read-write conflict

```
atomic {  
    ... = *z;  
    ...  
    work(...);  
    ...  
}
```

Zombie Transactions:

- Conflicted transaction (i.e., doomed to fail) not yet detected
- Due to lazy conflict detection
- Worsened by long transactions

TM Challenges – Zombie Transactions

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

read-write conflict

zombie transaction

```
atomic {  
    *x = 1;  
    *y = 1;  
    work(...);  
    ...  
}
```

read-write conflict

zombie transaction

```
atomic {  
    ... = *z;  
    ...  
    work(...);  
    ...  
}
```

Zombie Transactions:

- Conflicted transaction (i.e., doomed to fail) not yet detected
- Due to lazy conflict detection
- Worsened by long transactions

TM Challenges

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

TM Challenges

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

**Assume eager version management
with lazy conflict detection**

TM Challenges

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

```
atomic {  
    *x = 1;  
    *y = 1;  
    work(...);  
    ...  
}
```

TM Challenges

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

reads *x=1 and *y=1

```
atomic {  
    *x = 1;  
    *y = 1;  
    work(...);  
    ...  
}
```

ok if commits first

TM Challenges – False Conflicts

```
atomic {  
    ... = *x;  
    ... = *y;  
    *z = 2;  
    work(...);  
    if (...) {...}  
    else {...}  
    ...  
}
```

reads *x=1 and *y=1

```
atomic {  
    *x = 1;  
    *y = 1;  
    work(...);  
    ...  
}
```

ok if commits first

False Conflicts:

- Safe (atomic and isolated) executions can yield rollbacks
- Due to conservative version management (i.e., read/write sets)

TM Challenges

```
node *top;
```

```
atomic {  
    node *old = top;  
    if (top)  
        top = top->next;  
    ...  
}
```

```
atomic {  
    node *old = top;  
    if (top)  
        top = top->next;  
    ...  
}
```

Any potential problems?

TM Challenges

```
node *top;
```

```
atomic {  
    node *old = top;  
    if (top)  
        top = top->next;  
    ...  
}
```

```
atomic {  
    node *old = top;  
    if (top)  
        top = top->next;  
    ...  
}
```

**Assume eager version management
with lazy conflict detection**

TM Challenges

```
node *top;
```

```
atomic {  
    node *old = top;  
    if (top)  
        top = top->next;  
    ...  
}
```

```
atomic {  
    node *old = top;  
    if (top)  
        top = top->next;  
    ...  
}
```

seg fault!

TM Challenges

```
unsigned int end = 0;  
unsigned int start = 0;
```

```
atomic {  
    for(i = 0; i < end-start; i++)  
        ...  
}
```

```
atomic {  
    end = 99;  
    start = 1;  
    for(...)  
}
```

Any potential problems?

TM Challenges

```
unsigned int end = 0;  
unsigned int start = 0;
```

```
atomic {  
    for(i = 0; i < end-start; i++)  
        ...  
}
```

```
atomic {  
    end = 99;  
    start = 1;  
    for(...)  
}
```

**Assume eager version management
with lazy conflict detection**

TM Challenges

```
unsigned int end = 0;  
unsigned int start = 0;
```

```
atomic {  
    for(i = 0; i < end-start; i++)  
        ...  
}
```

(unsigned) 0 – 1

conflict won't get
detected until much later!

```
atomic {  
    end = 99;  
    start = 1;  
    for(...)  
}
```



TM Challenges

```
atomic {  
    queueX->enqueue(51);  
    while(queueY->empty());  
    ...  
}
```

```
atomic {  
    queueY->enqueue(12);  
    while(queueX->empty());  
    ...  
}
```

Any potential problems?

TM Challenges

```
atomic {  
    queueX->enqueue(51);  
    while(queueY->empty());  
    ...  
}
```

```
atomic {  
    queueY->enqueue(12);  
    while(queueX->empty());  
    ...  
}
```

**Assume lazy version management
with lazy conflict detection**

TM Challenges

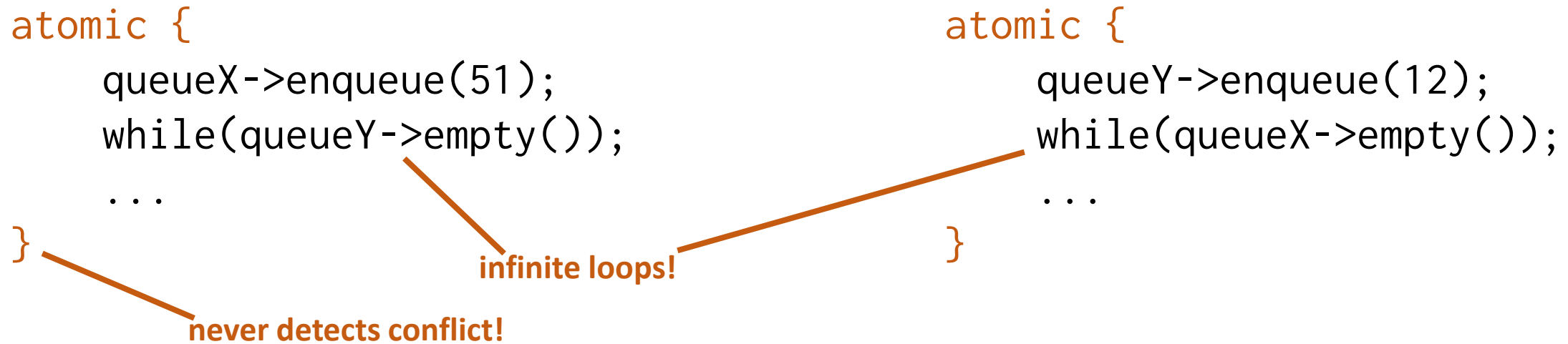
```
atomic {  
    queueX->enqueue(51);  
    while(queueY->empty());  
    ...  
}
```

never detects conflict!

infinite loops!

```
atomic {  
    queueY->enqueue(12);  
    while(queueX->empty());  
    ...  
}
```

TM Challenges – False Synchronization



False Synchronization:

- Conflicts are not necessarily prevented nor eventually detected
- Due to lazy conflict detection
- Transactions $\neq$ critical sections (i.e., atomicity vs. mutual exclusion)

Eager vs. Lazy Conflict Detection

When to use Eager

- Few load and store instructions (avoid overhead of frequent eager checks)
- High conflict rate expected (catch conflicts as early as possible)

vs

When to use Lazy

- Small memory footprint within transactions (avoid complexity of read/write set matching at commit)
- **Low conflict rate expected (progress faster since conflicts are unlikely)**

key motivation of transactional memory

Eager vs. Lazy Conflict Detection

When to use Eager

- Few load and store instructions (avoid overhead of frequent eager checks)
- High conflict rate expected (catch conflicts as early as possible)
- **Need for simpler, safer execution for programmer**

vs

When to use Lazy

- Small memory footprint within transactions (avoid complexity of read/write set matching at commit)
- **Low conflict rate expected (progress faster since conflicts are unlikely)**

key motivations of transactional memory
conflict with each other

